

# 2차 회의록

- 🕒

생성 일시

2026년 5월 7일 오전 8:58
- 👥

참석자

은

은 지

우석

정우석

유빈

유빈

서은신

서은신
- ≡

비고

비어 있음
- ✓

속성 3개 더 보기

출결현황

숙제

논의사항

진행 상황 점검

1차 진행 상황(9시)

강사님 피드백(10시)

3차 진행 상황(15시 반)

강사님 피드백 (16시)

워치



## 회의 요약

- 리스트

## 출결현황

- ✓

장지은
- ✓

정우석
- ✓

서은산
- ✓

이유빈 (조퇴: 16:00)

## 숙제

- 리스트

## 논의사항

- 리스트

## 진행 상황 점검

### 1차 진행 상황(9시)

- 지은:
- 우석:
- 은신:
- 유빈:

### 강사님 피드백(10시)

- 리스트

### 3차 진행 상황(15시 반)

- 지은:
- 우석:
- 은신:
- 유빈:

### 강사님 피드백 (16시)

- 리스트

## 위치

**Wear OS PTT API:** 직접적인 워키토키 앱 소스 코드는 공개되어 있지 않지만, Wear OS에서는 마이크 입력과 데이터 전송을 위한 **DataLayer API**를 사용해 유사한 기능을 구현할 수 있습니다.

## ▼ DataLayer API

안드로이드 Wear OS 개발에서 **DataLayer API**는 핸드폰(Handheld)과 워치(Wearable) 간의 데이터를 동기화하고 통신하기 위한 핵심 수단입니다. Google Play 서비스의 일부인 `Wearable.getDataClient` 등을 통해 구현됩니다.

DataLayer를 효과적으로 사용하기 위한 주요 구성 요소와 구현 단계를 정리해 드립니다.

## 1. 주요 구성 요소 (Components)

DataLayer API는 크게 세 가지 방식으로 데이터를 주고받습니다.

| 구성 요소                           | 특징                                     | 주 용도                              |
|---------------------------------|----------------------------------------|-----------------------------------|
| <b>DataClient</b><br>(DataItem) | 데이터가 로컬 스토리지에 저장되며, 연결 시 자동 동기화됩니다.    | 설정값, 사용자 프로필, 상태 유지               |
| <b>MessageClient</b>            | 짧은 메시지를 전송합니다.<br>(Fire-and-forget 방식) | 원격 실행, 특정 액션 트리거<br>(예: 음악 재생/정지) |
| <b>NodeClient</b>               | 연결된 기기(노드)의 상태 및 정보를 확인합니다.            | 연결 여부 확인, 특정 기기 ID 추출             |

## 2. 개발 환경 설정

`build.gradle (Module: app / wear)` 파일에 Wearable 라이브러리를 추가해야 합니다. 양쪽 모듈 모두 동일한 패키지명을 가져야 통신이 가능합니다.

Gradle

```
dependencies {  
    implementation 'com.google.android.gms:play-services-wearable:18.1.0'  
}
```

## 3. 핵심 구현 단계

## 단계 1: 클라이언트 인스턴스 생성

데이터 전송이나 수신을 위해 각 기기에서 클라이언트를 초기화합니다.

Kotlin

```
val dataClient = Wearable.getDataClient(context)
val messageClient = Wearable.getMessageClient(context)
```

## 단계 2: 데이터 전송 (PutDataItem)

데이터를 보낼 때는 `PutDataMapRequest` 를 사용합니다. 이 데이터는 일종의 "지도 (Map)" 형태로 저장되며, 경로(Path)를 통해 식별됩니다.

Kotlin

```
val request = PutDataMapRequest.create("/user_profile").apply {
    dataMap.putString("name", "John Doe")
    dataMap.putInt("age", 25)
    // 데이터 변경 시간을 추가하여 매번 동기화가 발생하도록 강제함
    dataMap.putLong("timestamp", System.currentTimeMillis())
}
val putDataTask = dataClient.putDataItem(request.asPutDataRequest())
```

## 단계 3: 데이터 수신 (OnDataChangedListener)

데이터 변화를 감지하려면 `DataClient.OnDataChangedListener` 를 구현하거나, 백그라운드에서도 수신하려면 `WearableListenerService` 를 상속받은 서비스를 등록해야 합니다.

Kotlin

```

override fun onChanged(dataEvents: DataEventBuffer) {
    for (event in dataEvents) {
        if (event.type == DataEvent.TYPE_CHANGED) {
            val path = event.dataItem.uri.path
            if (path == "/user_profile") {
                val dataMap =
                    DataMapItem.fromDataItem(event.dataItem).dataMap
                val name = dataMap.getString("name")
                // UI 업데이트 로직
            }
        }
    }
}

```

## 4. 주의사항 및 팁

- **동일 패키지명 & 서명:** 핸드폰 앱과 워치 앱의 패키지 이름(`applicationId`)과 서명(Signing)이 반드시 일치해야 보안 통신이 허용됩니다.
- **지연 발생 가능성:** DataItem은 동기화 방식이므로 배터리 최적화를 위해 즉시 전송되지 않을 수 있습니다. 즉각적인 반응이 필요하다면 `MessageClient`를 권장합니다.
- **용량 제한:** DataItem의 크기는 약 100KB로 제한됩니다. 대용량 파일(이미지 등)은 `Asset` 객체를 사용하여 전송해야 합니다.
- **노드 확인:** 메시지를 보낼 때는 `NodeClient`를 통해 현재 연결된 노드 ID를 먼저 가져와야 정확한 대상에게 전송할 수 있습니다.

워키토키와 같은 실시간 음성 전송의 경우, `MessageClient`로 연결 상태를 제어하고, 실제 음성 바이트 데이터는 `ChannelClient`를 사용하는 것이 스트리밍 효율 면에서 더 유리할 수 있습니다.

## ▼ 기초 프로젝트 설정

```
dependencies { // 기본 AndroidX 및 UI 구성 요소
    implementation("androidx.appcompat:appcompat:1.7.1") // 안드로이드
    앱의 하위 호환성
    implementation("com.google.android.material:material:1.13.0")
    implementation("androidx.constraintlayout:constraintlayout:2.2.1")
    // Firebase(bom) // bom: 버전 통합 관리
    implementation(platform("com.google.firebase:firebase-bom:33.1.0"))
    implementation("com.google.firebase:firebase-auth-ktx")
    implementation("com.google.firebase:firebase-firestore-ktx")
    implementation("com.google.firebase:firebase-database-ktx") //
    Google Maps & Location implementation("com.google.android.gms:play-
    services-maps:18.2.0") implementation("com.google.android.gms:play-
    services-location:21.2.0") // Compose용 Google Maps (사용 시)
    implementation("com.google.maps.android:maps-compose:4.3.3") //
    Wearable DataLayer (폰-위치 통신)
    implementation("com.google.android.gms:play-services-
    wearable:19.0.0") // Wear OS 전용 UI 라이브러리
    implementation("androidx.wear.compose:compose-material:1.3.1")
    implementation("androidx.wear.compose:compose-navigation:1.3.1") //
    위치용 위치 서비스 최적화
    implementation("com.google.android.gms:play-services-
    location:21.2.0") // Compose
    implementation("androidx.compose.material:material-icons-extended")
    //kako 공유 implementation("com.kakao.sdk:v2-share:2.20.1") }
```

```

<!-- 필수 권한 설정 --> <uses-permission
android:name="android.permission.INTERNET" /> <uses-permission
android:name="android.permission.ACCESS_NETWORK_STATE" /> <uses-
permission android:name="android.permission.ACCESS_FINE_LOCATION"
/> <uses-permission
android:name="android.permission.ACCESS_COARSE_LOCATION" /> <uses-
permission
android:name="android.permission.ACCESS_BACKGROUND_LOCATION" />
<uses-permission android:name="android.permission.RECORD_AUDIO" />
<uses-permission
android:name="android.permission.FOREGROUND_SERVICE" /> <uses-
permission
android:name="android.permission.FOREGROUND_SERVICE_LOCATION" />
<uses-permission
android:name="android.permission.FOREGROUND_SERVICE_MICROPHONE" />

```

```

<application ...> <!-- 테스트키:
AIzaSyDg3wiqr1hSD1nfs_kXBPzt1EHc0570kI --> <meta-data
android:name="com.google.android.geo.API_KEY"
android:value="MAPS_API_KEY" /> <queries> <package
android:name="com.kakao.talk" /> </queries> <!-- GlobalApplication
라인 추가 --> <application android:name=".GlobalApplication" ...>
<service android:name=".service.MyDataLayerListenerService"
android:exported="true"> <intent-filter> <action
android:name="com.google.android.gms.wearable.DATA_CHANGED" />
<action
android:name="com.google.android.gms.wearable.MESSAGE_RECEIVED" />
<data android:scheme="wear" android:host="*"
android:pathPrefix="/walkie_talkie" /> <data android:scheme="wear"
android:host="*" android:pathPrefix="/location" /> </intent-filter>
</service> </application>

```



```
// settings.gradle.kts dependencyResolutionManagement {
repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
repositories { google() mavenCentral() // 카카오 SDK사용을 위해 아래
한 줄 추가 maven { url =
uri("https://devrepo.kakao.com/nexus/content/groups/public/") } } }
```

```
package com.example.finaltest.service import
com.google.android.gms.wearable.WearableListenerService //
DataLayer 작업을 수행 class MyDataLayerListenerService:
WearableListenerService() { }
```

```
package com.example.finaltest import android.app.Application import
com.kakao.sdk.common.KakaoSdk // 어플리케이션 초기화에 관련된 작업을
수행 class GlobalApplication : Application() { override fun
onCreate() { super.onCreate() // Kakao SDK 초기화 // 테스트키:
a98bcee5db7e539f6559d685aea78cfd // 카카오 디벨로퍼 홈페이지에 테스트
PC 해시키 등록되어 있지 않으면 테스트 불가(아마도?)
KakaoSdk.init(this, "NATIVE_APP_KEY") } }
```

## ▼ 이슈 목록

## ▼ 전송 비용과 속도

Wear OS의 **DataLayer API**를 통해 위치와 휴대폰 간에 음성 데이터를 주고받을 때 발생하는 지연 시간(Latency)은 **어떤 클라이언트를 사용하느냐와 블루투스 연결 상태**에 따라 크게 달라집니다.

결론부터 말씀드리면, 실시간 음성 스트리밍을 구현할 경우 **약 100ms ~ 300ms** 정도의 지연이 일반적이며, 최적화가 안 된 경우 1초 이상의 지연이 발생할 수도 있습니다.

| 단계       | 처리 내용                                   | 예상 소요 시간       |
|----------|-----------------------------------------|----------------|
| 녹음 및 버퍼링 | 마이크 신호를 디지털로 바꿔서 일정 크기로 모음              | 약 20ms ~ 40ms  |
| 코덱 인코딩   | 용량을 줄이기 위해 압축 (예: Opus)                 | 약 10ms ~ 30ms  |
| 블루투스 전송  | **DataLayer(ChannelClient)**를 통한 물리적 이동 | 약 50ms ~ 150ms |
| 디코딩 및 재생 | 압축 해제 후 스피커로 출력                         | 약 20ms ~ 80ms  |

## 3. 변수에 따른 변화

- **0.1초(100ms)에 가까워지는 경우:** **ChannelClient**를 사용하고, 오디오 샘플링 레이트를 낮추고(16kHz 등), 전송 패킷 크기를 아주 작게 최적화했을 때 가능합니다.
- **0.3초(300ms) 이상으로 늘어나는 경우:** 블루투스 주변 간섭이 심하거나, **MessageClient**를 사용해 데이터를 덩어리째 보낼 때, 혹은 폰의 CPU가 다른 작업으로 바쁠 때 발생합니다.

▼ 음성 메세지 저장

단순히 실시간 스트리밍만 고집할 경우, 상대방이 네트워크가 불안정하거나 앱을 종료한 상태라면 서비스의 효용성이 급격히 떨어지기 때문입니다. 이를 구현하기 위한 기술적 고려사항과 비용 효율적인 전략을 정리해 드립니다.